

1. Einleitung und Messaufbau

1.1 System

Tabelle 1.1: Messsystem

Komponente	Modell	Eckdaten
GPU	NVIDIA GeForce RTX 3070	Ampere, SM 8.6, 46 SMs, 8 GB GDDR6, 256-bit, 1755 MHz
CPU	AMD Ryzen 7 3800X	8 Kerne / 16 Threads, seriell und OpenMP
Toolchain	CUDA 12.0	Linux-nvcc unter WSL2 (Ubuntu)

Build-Weg: Kompiliert und gemessen wurde unter WSL2. Der native Windows-nvcc ist auf meinem Rechner ständig mit `Host compiler targets unsupported OS` abgebrochen; über den WSL-GPU-Passthrough lief dieselbe RTX 3070 dagegen problemlos.

1.2 Messmethodik

Gemessen habe ich die reine Kernel-Zeit über `cudaEvents`, Host-Device-Transfers sind nicht enthalten. Vor jeder Messung laufen drei Warm-up-Durchläufe, damit CUDA-Context, JIT und Taktboost nicht in die Messung fallen. Angegeben ist der Median aus zehn Wiederholungen, auf der CPU aus zwei bis drei. Die Korrektheit prüfe ich pro Messung an 1024 Stichproben gegen eine skalare CPU-Referenz.

1.3 Profiler-Einschränkung

Ein direkter Nachweis über `ncu`, etwa der Warp-Execution-Efficiency oder der achieved Occupancy, war unter WSL2 nicht möglich. Die angegebenen Occupancy-Werte stammen deshalb aus dem Belegungsmodell (`cudaOccupancyMaxActiveBlocksPerMultiprocessor`) zusammen mit der Launch-Konfiguration und nicht aus einer Profiler-Messung.

2. SIMT-Ausführungsmodell und Warp-Divergenz

2.1 Kernel-Design

Jeder Thread bearbeitet ein Element und iteriert darauf eine FMA-Kette $x = x \cdot a_p + b_p$ über k Iterationen (Standard: $k = 4096$, also 8192 FLOPs pro Element). Pro Element fallen ein Lade- und ein Speicherzugriff an. Bei einer arithmetischen Intensität von etwa 1024 FLOP/Byte ist der Kernel damit rechenlimitiert. Der Speicherzugriff sorgt außerdem dafür, dass der Compiler die Rechnung nicht als toten Code entfernt (siehe `aufgabe1/src/divergence_bench.cu`).

Ein `switch` wählt einen von d Pfaden mit $d \in \{1, 2, 4, 8, 16, 32\}$. Jeder Pfad ist eine eigene FMA-Schleife mit eigenen Konstanten a_p, b_p , sodass der Compiler die Pfade nicht zusammenlegen kann. Jeder Thread führt unabhängig von d exakt k Iterationen aus, die Gesamtrechenarbeit ist also für alle Divergenzgrade gleich. Es ändert sich nur, wie viele *verschiedene* Pfade innerhalb eines 32er-Warps auftreten. Gemessen wird damit die Divergenz-Serialisierung allein, ohne Last-Ungleichgewicht.

Tabelle 2.1: Varianten des Divergenz-Benchmarks

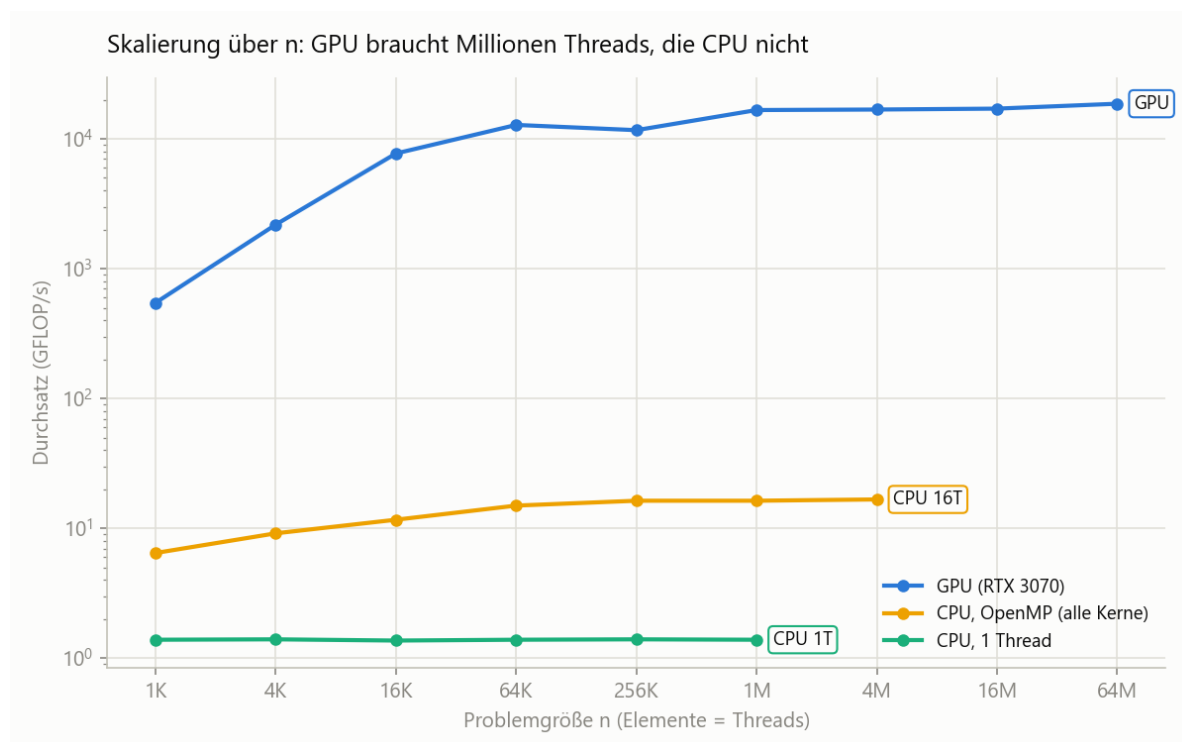
Variante	Pfadwahl	Eigenschaft
gpu-intra	<code>threadIdx.x % d</code>	d Pfade innerhalb jedes Warps (echte Divergenz)
gpu-warpuniform	<code>(threadIdx.x / 32) % d</code>	gleiche Pfadverteilung, aber warp-einheitlich
cpu / cpu-omp	<code>i % d</code>	seriell bzw. mit OpenMP über 16 Threads

Jeder Pfad konvergiert gegen einen anderen Fixpunkt $b_p/(1 - a_p)$. Der Korrektheitscheck weist damit zugleich nach, dass jedes Element den vorgesehenen Pfad genommen hat. Der Durchsatz ergibt sich als $\text{GFLOP/s} = 2kn/t$, da eine FMA zwei FLOPs entspricht.

2.2 Skalierung über die Problemgröße

Der GPU-Durchsatz steigt von 546 GFLOP/s bei $n = 1024$ über gut zwei Größenordnungen und läuft dann in ein Plateau. Der Knick liegt bei $n \approx 64\text{K}$ mit 12,8 TFLOP/s, das sind rund 69 % des Maximums. Ab $n \approx 1\text{M}$ liegt der Durchsatz bei 16,6 TFLOP/s, der Höchstwert von 18,6 TFLOP/s wird bei $n = 64\text{M}$ erreicht und entspricht etwa 92 % des theoretischen FP32-Peaks von rund 20,3 TFLOP/s. Zwischen kleinstem und größtem n liegt damit ein Faktor 34, bei identischer Arbeit pro Thread.

Auf der CPU zeigt sich das umgekehrte Bild. Mit OpenMP über alle 16 Threads liegt der Durchsatz schon bei kleinem n auf seinem Endwert von etwa 16,7 GFLOP/s, der serielle

Abbildung 2.1: Durchsatz über der Problemgröße n

Lauf bleibt über den gesamten Bereich bei 1,39 GFLOP/s. Im Plateau ist die GPU damit etwa 1100-mal schneller als die 16-Thread-CPU und etwa 13 400-mal schneller als ein einzelner Kern.

Die RTX 3070 hat 46 SMs mit je bis zu 1536 residenten Threads, kann also etwa 70 000 Threads gleichzeitig halten und benötigt ein Mehrfaches davon, um Latenzen zu überdecken. Unterhalb dieser Schwelle bleiben SMs unbeschäftigt und der Durchsatz skaliert etwa linear mit n . Bei der CPU tritt dieser Anlauf nicht auf, da bereits eine Handvoll Threads für die volle Auslastung genügt.

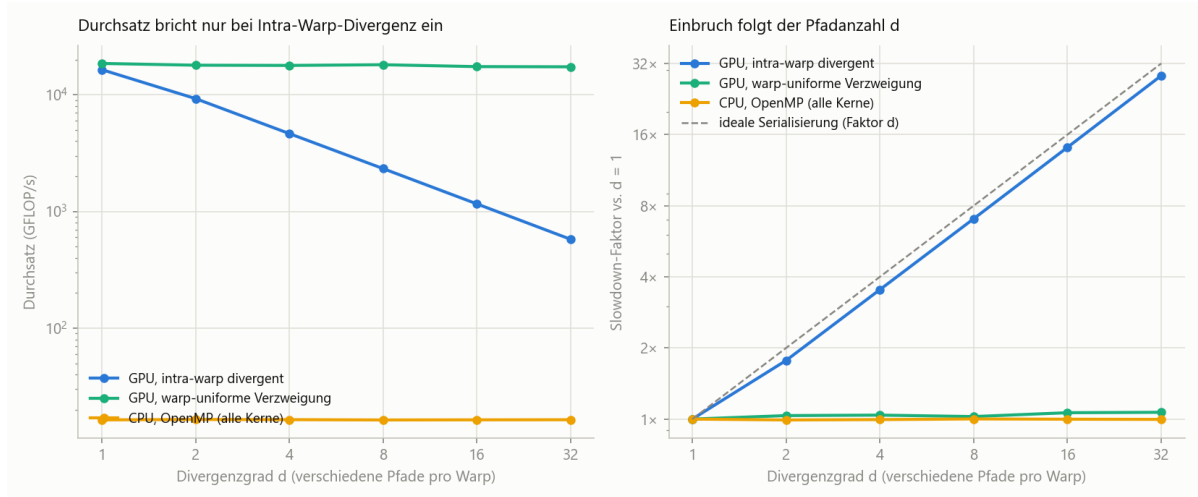
2.3 Warp-Divergenz

Bei `gpu-intra` fällt der Durchsatz mit steigendem d entlang der idealen $1/d$ -Kurve, von 16,4 TFLOP/s bei $d = 1$ auf 577 GFLOP/s bei $d = 32$.

Tabelle 2.2: Slowdown von `gpu-intra` gegenüber $d = 1$

d	1	2	4	8	16	32
Slowdown	1,00×	1,77×	3,53×	7,04×	14,1×	28,4×

Der ideale Faktor wäre d selbst. Die Unterschreitung bei $d = 32$ (28,4 statt 32) geht auf feste Overheads und eine geringe Überlappung durch den Scheduler zurück. Die beiden Kontrollkurven bleiben flach: `gpu-warpuniform` liegt über den gesamten Sweep zwischen 17,3 und 18,5 TFLOP/s bei einer Schwankung von höchstens 1,07×, die CPU mit OpenMP bei 16,4 GFLOP/s mit höchstens 1,01×.

Abbildung 2.2: Durchsatz über dem Divergenzgrad d

Der Durchsatzverlust hängt damit an der Pfadanzahl. Treten d verschiedene Pfade pro Warp auf, führt die Hardware sie nacheinander aus und der Durchsatz fällt auf etwa $1/d$. Nicht die Verzweigung selbst kostet: Dieselbe `switch`-Anweisung mit warp-einheitlicher Pfadwahl bleibt ohne Einbruch. Dass die Warp-Effizienz sinkt, also pro Instruktion nur noch etwa $32/d$ Lanes aktiv sind, lässt sich am Durchsatz ablesen, denn ein Verlauf entlang der $1/d$ -Linie entspricht einer Warp-Ausführungseffizienz von $1/d$. Der Profiler stand dafür unter WSL2 nicht zur Verfügung; Kontrollexperiment und Kurvenform decken den Mechanismus ab.

2.4 Ursache der Serialisierung

Ein Warp ist die Ausführungseinheit des SIMT-Modells. 32 Threads teilen sich einen Instruktionsstrom und werden im Gleichschritt auf dieselben Rechenwerke gegeben. Nehmen Threads eines Warps verschiedene Zweige, lassen sich nicht beide anteilig bedienen. Die Zweige werden nacheinander ausgeführt, wobei jeweils die Threads maskiert werden, die den gerade laufenden Zweig nicht gewählt haben (Active Mask und Reconvergence). Bei d disjunkten, gleich teuren Zweigen sind d Durchläufe nötig, in denen jeweils nur $32/d$ der Lanes nützliche Arbeit leisten. Bei $d = 32$ arbeitet pro Takt nur eine Lane. Daran ändert auch das Independent Thread Scheduling seit Volta nichts Grundsätzliches. Es erlaubt eine feinere Verschränkung der Zweige und verhindert bestimmte Deadlocks, gibt aber weiterhin pro Takt nur eine Instruktion für die aktive Teilmenge des Warps aus.

Das ist der Preis der Rechendichte. Weil sich 32 Lanes Fetch, Decode und Scheduling teilen, passen zehntausende Threads auf einen Chip; die Kontrolllogik, die Verzweigungen schnell macht, fehlt dafür.

Auf der CPU hat jeder Kern einen eigenen Program Counter und ein eigenes Frontend. Threads auf verschiedenen Kernen sind vollständig unabhängig, Divergenz zwischen ihnen gibt es als Konzept nicht. Innerhalb eines Kerns kostet eine vorhersagbare Verzweigung dank Sprungvorhersage fast nichts. Ein Muster wie `i % d` lernt der Prädiktor sofort, der Kern spekuliert korrekt am Zweig vorbei und die Pipeline bleibt gefüllt. Kosten entstehen nur bei einer Fehlvorhersage, etwa 15 bis 20 Takte für den Pipeline-Flush, was in diesem Benchmark kaum vorkommt.

Sobald CPU-Code mit SIMD (AVX) vektorisiert wird, taucht dasselbe Problem als Maskierung wieder auf, denn divergente Lanes eines Vektorregisters müssen ebenfalls beide Zweige rechnen. SIMT auf der GPU und SIMD auf der CPU unterscheiden sich hier nur graduell. Der eigentliche Unterschied liegt zwischen unabhängigen Kernen mit Sprungvorhersage und Lanes im Gleichschritt.

3. Speicherzugriff: Latenz verstecken und Bandbreite

3.1 Kernel-Design

Der Streaming-Kernel `b[i] = a[idx(i)] * c` führt pro Element einen Lade- und einen Speicherzugriff sowie eine Multiplikation aus. Bei einer arithmetischen Intensität von 0,125 FLOP/Byte ist er speicherlimitiert. Als Referenz dient die theoretische Peak-Bandbreite von 448 GB/s, berechnet als $2 \cdot \text{Speichertakt} \cdot \text{Busbreite} / 8$ aus `cudaDeviceProp`, also mit derselben Formel wie `deviceQuery`. Variiert wird ausschließlich die Abbildung des Leseindex (siehe `aufgabe2/src/bandwidth_bench.cu`).

Tabelle 3.1: Untersuchte Zugriffsmuster

Muster	Index	Charakteristik
coalesced	$\text{idx} = i$	benachbarte Threads lesen benachbarte Adressen (Stride 1)
strided	$\text{idx} = (i \bmod m) \cdot s + \lfloor i/m \rfloor$	Transpose-Bijektion, deckt <code>a</code> genau einmal ab
gather	$\text{idx} = \text{perm}[i]$	zufällige Permutation, vollständig gestreut

Pro Element werden 4 Byte gelesen und 4 Byte geschrieben, die effektive Bandbreite ist damit $8n/t$. Beim gather-Muster kommt der Index-Strom mit weiteren $4n$ Byte hinzu, und für jedes genutzte 4-Byte-Wort wird ein ganzer 32-Byte-Sektor geladen. Die tatsächliche DRAM-Aktivität liegt dort also über der ausgewiesenen nützlichen Bandbreite. Für den Vergleich der Muster gebe ich durchgehend $8n/t$ an.

3.2 Einfluss des Zugriffsmusters

Das coalesced-Muster erreicht 368 GB/s, also 82 % der theoretischen Peak-Bandbreite. Bei gestreutem Zugriff fällt der Durchsatz ab: strided mit $k = 8$ auf 86 GB/s (19 %), zufälliger gather auf 34 GB/s (7,6 %), bei jeweils gleicher Nutzdatenmenge.

Der Stride-Sweep zeigt den Übergang. Von Stride 1 mit 374 GB/s fällt die Bandbreite über 265 GB/s ($k = 2$) und 162 GB/s ($k = 4$) auf ein Plateau um 87 GB/s ab Stride 8 und weiter auf rund 64 GB/s bei Stride ≥ 64 .

Bei coalesced-Zugriff liegen die Adressen der 32 Threads eines Warps in wenigen zusammenhängenden Cache-Zeilen. Sie werden zu wenigen breiten Speichertransaktionen zusammengefasst, jeder geladene 32-Byte-Sektor bzw. jede 128-Byte-Zeile wird voll genutzt. Mit wachsendem Stride verteilen sich die Zugriffe auf mehr Sektoren, von denen jeweils nur ein Teil gebraucht wird. Für dieselben Nutzbytes sind dann mehr Transaktionen nötig und der Durchsatz sinkt. Überschreitet der Stride die Sektorgranularität von

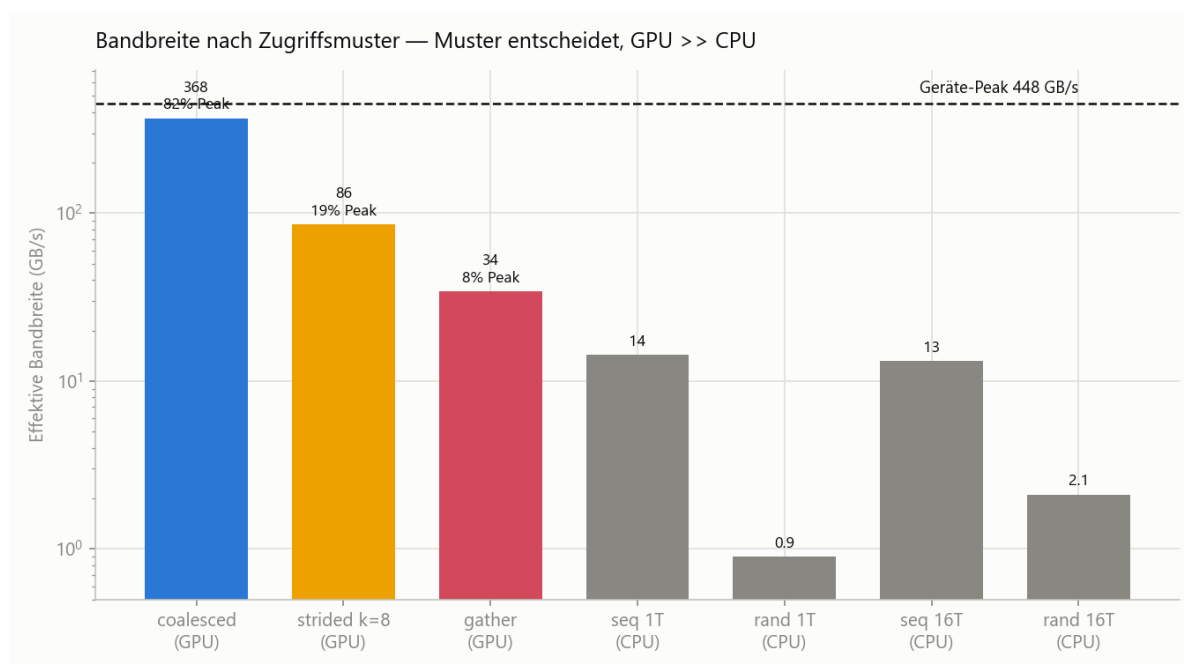


Abbildung 3.1: Effektive Bandbreite nach Zugriffsmuster

32 Byte, also acht floats, braucht praktisch jeder Thread eine eigene Transaktion; darauf geht das Plateau zurück. Die weitere Abnahme bei sehr großen Strides stammt aus TLB- und Seiten-Effekten. Beim zufälligen gather fehlt jede räumliche Lokalität, die genutzte Bandbreite liegt unter einem Zehntel des coalesced-Werts.

3.3 Latenz verstecken über Occupancy

Gemessen wurde der coalesced-Kernel bei Blockgröße 256 mit maximal sechs Blöcken pro SM, entsprechend 48 Warps und 100 % Occupancy auf dieser Karte. Mit acht residenten Warps pro SM (17 % Occupancy) liegt der Kernel bei 243 GB/s. Der Durchsatz steigt mit der Zahl der Warps und liegt ab etwa 24 Warps (50 % Occupancy) bei rund 380 GB/s; weitere Warps bringen keinen Gewinn mehr.

Ein einzelner DRAM-Zugriff kostet mehrere hundert Takte, und solange ein Warp auf seine Daten wartet, ist er blockiert. Diese Wartezeit wird nicht durch einen großen Cache überbrückt, sondern dadurch, dass der Warp-Scheduler in jedem Takt einen anderen, gerade lauffähigen Warp auswählt. Sind zu wenige Warps resident, fehlen dem Scheduler die lauffähigen Kandidaten. Die Speicher-Stalls werden dann sichtbar und der Durchsatz sinkt, bei 17 % Occupancy auf 65 % der gesättigten Bandbreite. Ab genügend Warps sind immer Zugriffe unterwegs, die Latenz ist überdeckt und der Kernel nicht mehr latenz-, sondern bandbreitenlimitiert. Weitere Warps wirken sich deshalb nicht mehr aus.

3.4 Zwei Latenz-Strategien im Vergleich

Auf einem CPU-Kern wird die Speicherlatenz über Cache-Hierarchie und Hardware-Prefetcher überdeckt. Beim sequenziellen Streaming erkennt der Prefetcher das Muster und holt die nächsten Cache-Zeilen im Voraus, der Kern erreicht mit etwa 14 GB/s annähernd seine DRAM-Bandbreite für diese Operation. Beim zufälligen Zugriff greifen weder

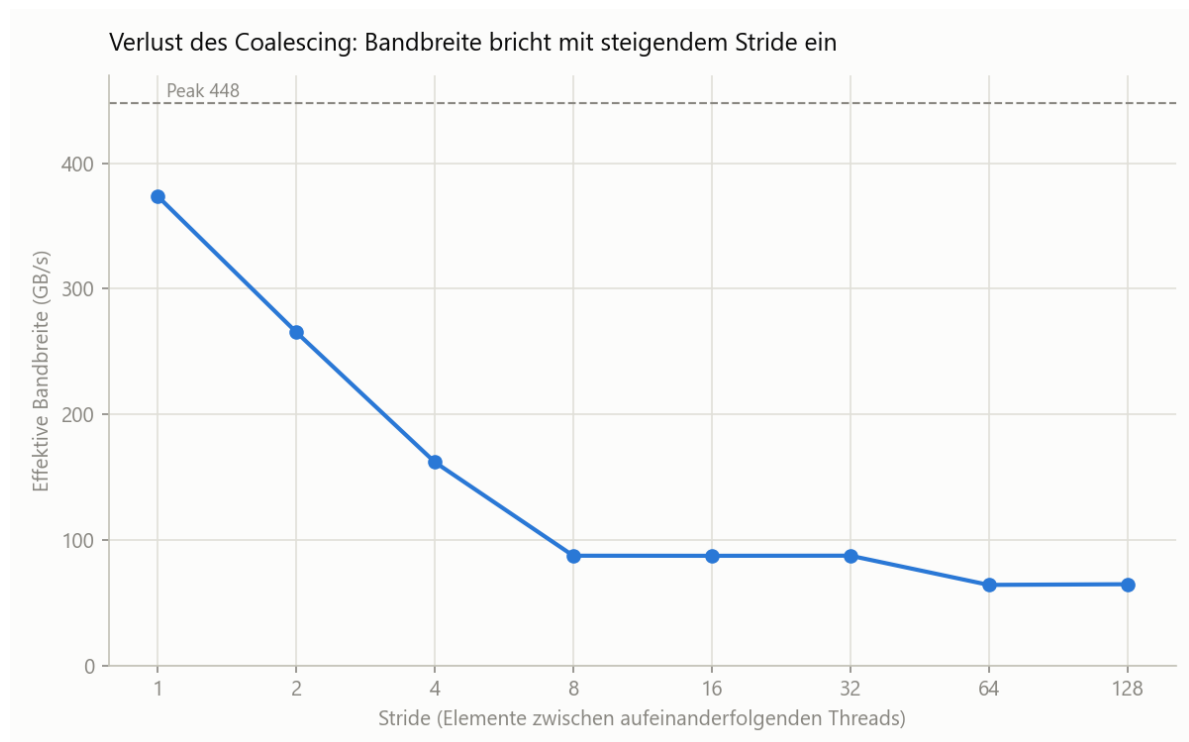


Abbildung 3.2: Effektive Bandbreite über dem Stride

Tabelle 3.2: CPU-Bandbreite bei sequenziellem und zufälligem Zugriff

Konfiguration	sequenziell	zufällig	Penalty
CPU, 1 Thread	14,4 GB/s	0,90 GB/s	16×
CPU, 16 Threads (OpenMP)	13,2 GB/s	2,11 GB/s	6,3×

Prefetching noch Lokalität. Fast jeder Zugriff ist ein Cache- und oft auch ein TLB-Miss, der Kern wartet die volle Latenz ab und der Durchsatz fällt um das 16-fache. Mehr Threads helfen dem Zufallsfall etwas, weil mehrere Kerne parallel auf ausstehende Misses warten (Memory-Level-Parallelismus), bleiben aber weit unter dem sequenziellen Wert. Im sequenziellen Fall bringt OpenMP kaum etwas, da schon ein einzelner Kern durch die Speicheranbindung limitiert ist.

Auf der GPU gibt es weder Prefetcher noch großen Cache, dafür zehntausende gleichzeitige Threads. Der gather-Fall mit 34 GB/s fällt relativ betrachtet weniger stark ab als auf der CPU, nämlich um Faktor 11 gegenüber dem eigenen coalesced-Wert statt um 16, weil die vielen Warps auch erfolglose Zugriffe überlappen. Schlechte Lokalität kostet aber auch hier, da sie sich nicht über Caching auffangen lässt.

Für die Datenanordnung heißt das, dass die Threads eines Warps auf zusammenhängende Adressen zugreifen sollten. Praktisch: Structure-of-Arrays statt Array-of-Structures, Indizierung so, dass `threadIdx.x` die innerste Stride-1-Dimension adressiert, gather und scatter vermeiden oder vorher sortieren und umpacken, und genügend Parallelität bereitstellen. Dasselbe Programm liefert je nach Layout zwischen 34 und 368 GB/s, das ist ein Faktor 11 allein aus der Datenanordnung. Im coalesced-Fall entspricht das etwa dem 26-fachen der sequenziellen Streaming-Bandbreite der CPU.

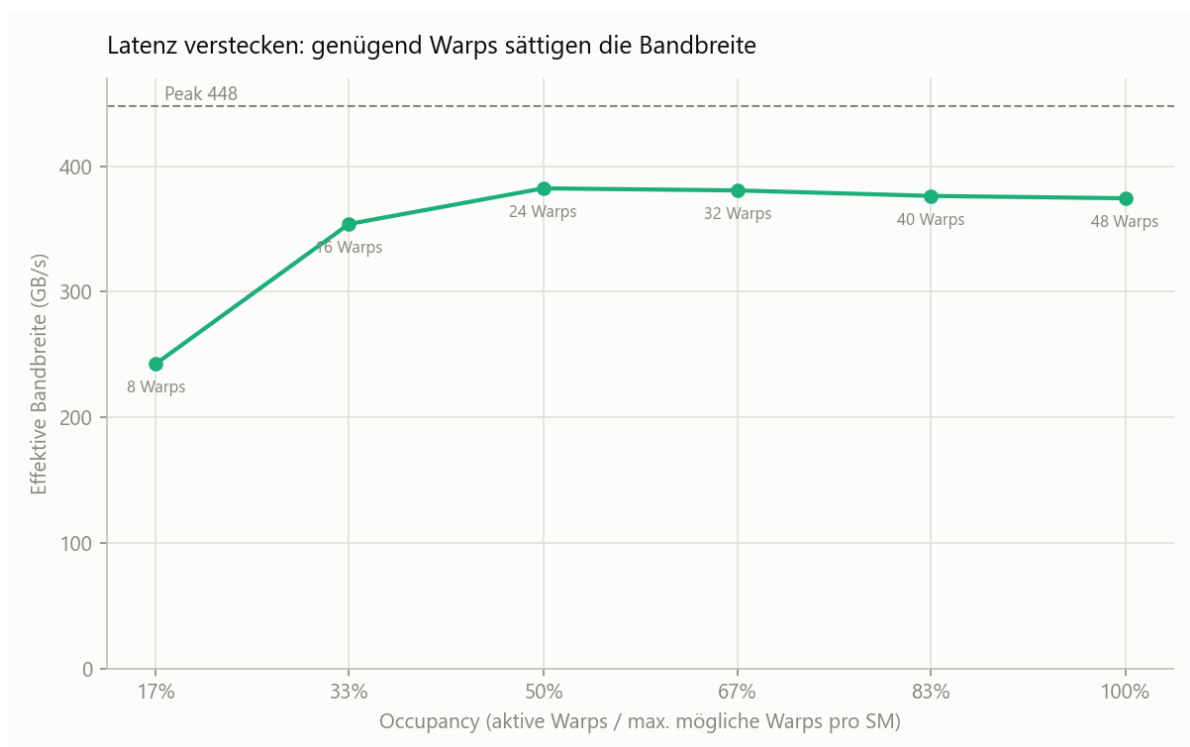


Abbildung 3.3: Effektive Bandbreite über der Occupancy

4. Fazit

Auf der GPU entsteht der Durchsatz aus der Zahl gleichzeitig laufender Warps, nicht aus der Geschwindigkeit einzelner Threads. Beide Aufgaben zeigen das an unterschiedlichen Stellen.

Die RTX 3070 erreicht ihren vollen Durchsatz von 18,6 TFLOP/s, also rund 92 % des FP32-Maximums, erst mit Millionen von Threads. Der Anlauf über den Faktor 34 bis zur Sättigung ab $n \approx 1\text{M}$ tritt auf der CPU nicht auf, die konstant bei 16,7 bzw. 1,39 GFLOP/s liegt. Im Plateau ist die GPU etwa 1100-mal schneller als die 16-Thread-CPU. Wird die Verzweigung warp-intern datenabhängig, fällt der Durchsatz mit $1/d$, bei $d = 32$ um Faktor 28,4. Das Kontrollexperiment `gpu-warpuniform` zeigt, dass die Serialisierung des Warps dafür verantwortlich ist und nicht die Verzweigung selbst.

Beim Speicherzugriff sättigt coalesced die Karte zu 82 % (368 GB/s), während strided auf 19 % und gather auf 7,6 % zurückfallen, jeweils bei gleicher Nutzdatenmenge. Zusätzlich müssen genügend Warps bereitstehen, um die Speicher-Stalls zu überbrücken; ab etwa 50 % Occupancy ist die Bandbreite ausgereizt.

In Aufgabe 1 lasten die Warps die Rechenwerke aus und werden durch Divergenz entwertet, in Aufgabe 2 überdecken sie die Speicherlatenz. Die CPU verhält sich in beiden Fällen anders: unabhängige Kerne mit Sprungvorhersage kennen keine Divergenz, Cache und Prefetcher machen sequenzielle Zugriffe schnell und zufällige langsam. Für GPU-Code folgt daraus, Threads eines Warps auf gleichförmige Kontrollflüsse und zusammenhängende Adressen auszulegen und genug Warps bereitzustellen, um Rechen- wie Speicherlatenz zu überdecken.